

LIBRARY MANAGEMENT SYSTEM

VITYARTHI FLIPPED COURSE PROJECT

Submitted By: SAKSHI
Registration No.: 25BA111058
Course: Object-Oriented Programming in Java
Date: 18 Sep 2026

1. Introduction & Problem Statement

Modern libraries require structured, reliable mechanisms to track physical and digital assets, manage member registrations, and monitor checkout lifecycles. Manual record-keeping leads to inaccuracies, lost items, and prolonged administrative friction.

The **Library Management System** solves these challenges by providing an automated, modular Java console application. It applies core Object-Oriented Programming (OOP) concepts to streamline inventory tracking, book issuance, and returns.

2. Functional Requirements

- **Book Inventory Module:** Allows adding, tracking, and listing book inventory items along with availability states.
- **Member Management Module:** Registers library patrons and stores member credentials securely.
- **Circulation Workflow:** Implements book borrowing and returning routines using strict validation checks via unique ISBN codes.
- **Interactive CLI Menu:** Provides a continuous loop interface for navigating administrative operations.

3. Non-Functional Requirements

Requirement	Specification
Performance	Instant O(N) array-list lookups for books and members under normal operating loads.
Maintainability	Modular architecture separating concerns across distinct classes (`Book`, `Member`, `Library`, `Main`).
Reliability	Robust error handling preventing unauthorized checkouts and invalid state transitions.
Usability	Clear text-based prompts guiding users seamlessly through each menu choice.

4. System Architecture & Design

The system follows a modular architectural pattern where business logic is separated from entity representations and user interface routines. Below is the structural overview of classes and relationships.

Class Structure & Composition

Composition Relationship: The `Library` class contains collections (`ArrayList`) of `Book` and `Member` objects, managing their interactions without tightly coupling individual entities.

- `Book.java` : Encapsulates attributes (`isbn`, `title`, `author`, `isAvailable`) and data-hiding methods.
- `Member.java` : Encapsulates patron profiles (`memberId`, `name`).
- `Library.java` : Core controller containing inventory logic, addition routines, and borrow/return validation algorithms.
- `Main.java` : Entry point housing the `main` method, user interaction loop, and `Scanner` input control.

5. Design Decisions & Rationale

Encapsulation was prioritized to protect book and member states from unauthorized direct modification. By declaring instance variables as `private` and exposing controlled getters and setters, data integrity is guaranteed throughout the application lifecycle.

6. Implementation & Testing Approach

The project was compiled and executed via command-line tools and verified within development environments such as IntelliJ IDEA. Testing involved unit validation of edge cases, such as attempting to borrow already checked-out books or searching for non-existent ISBNs.

7. Challenges Faced & Learnings

Managing object references across multiple classes required careful attention to encapsulation boundaries. This project reinforced practical command of ArrayList collections, constructor initialization, and control flow in Java.

8. Future Enhancements

- Integration of file persistence (`java.io` or database connectivity) to save records permanently across sessions.
- Implementation of inheritance to categorize member types (e.g., Student vs. Faculty members with custom borrowing limits).
- Development of a graphical user interface (GUI) using JavaFX or Swing.

9. References

1. Oracle Java Documentation - The Java Tutorials.
2. VITyarthi Course Guidelines on Object-Oriented Programming Projects.
3. Effective Java by Joshua Bloch.